Prior to Django 4.2

The `STORAGES` configuration value was introduced in Django 4.2. Prior to that, a different configuration value was necessary:

```
# RiffMates/RiffMates/settings.py
...
STATICFILES_STORAGE = \
    'django.contrib.staticfiles.storage.ManifestStaticFilesStorage'
```

This setting has been deprecated and will be removed in a future version of Django.

Using copies of static files in production means extra steps are required, and it complicates simple deployments, but it allows you to do fantastic things at scale. For example, you could have your static files served from an Amazon S3 bucket or a *content delivery network* (CDN). Both of these require custom handlers, and there are third-party packages that integrate the handling. For a bit more information and some pointers to other libraries, see https://docs.djangoproject.com/en/dev/howto/static-files/deployment/#staticfiles-from-cdn.

B.6 Other databases

Throughout the examples in this book, you've been using SQLite as your database, but Django supports four other databases out of the box:

- PostgreSQL
- MariaDB
- MySQL
- Oracle

There are also a number of third-party packages available that plug more database engines into your Django project. The ORM provides a common abstraction across all these databases, but it is just an abstraction. When you write code in a `Model`, Django is mapping your Python into SQL equivalents.

Different databases may handle the same SQL statement differently. For example, SQLite implements both the `CharField` and `TextField` as a text blob with no size limit, so it ignores the `max_length` attribute of the `CharField`. The `Model` still validates the length, but in a complicated set-up where multiple programs are sharing the database, you could end up with a `CharField` containing more characters than `max_length` value.

SQLite is the default database, with db.sqlite3 being the default database file. You've seen this file in your project, and if you were brave, you may have even deleted it. The configuration that determines this filename is in settings.py. The `DATABASES` configuration item specifies which database engine to use and what parameters that engine expects. The default Django puts in your settings.py when your project gets created looks like this: